Estructura de Datos y Complejidad Algorítmica

Análisis y Diseño de Algoritmos
Dividir y Conquistar

Dr. Pablo Ormeño-Arriagada

Ingeniería Civil Informática

24 de marzo de 2026

Objetivos de la Clase

- Comprender el análisis de algoritmos
- Evaluar la eficiencia computacional
- Introducir técnicas de diseño algorítmico
- Aplicar dividir y conquistar

¿Qué es un Algoritmo?

- Conjunto finito de instrucciones
- Resuelve un problema específico
- Debe ser correcto y eficiente

Importancia del Análisis

- Comparar soluciones
- Optimizar recursos
- Escalabilidad del sistema

Modelo de Computación

- Máquina RAM
- Operaciones básicas
- Costos unitarios

Complejidad Temporal

- Número de operaciones
- Depende del tamaño de entrada n
- Ejemplo: búsqueda lineal $O(n)$

Complejidad Espacial

- Memoria utilizada
- Variables auxiliares
- Importante en sistemas limitados

Notación Big-O

- Cota superior
- Describe el peor caso
- Ejemplo: $O(n^2)$

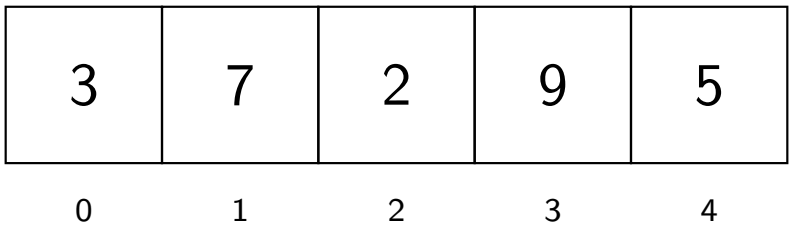
Otras Notaciones

- Ω (cota inferior)
- Θ (cota ajustada)

Casos de Análisis

- Mejor caso
- Caso promedio
- Peor caso

Representación de un Arreglo



- Acceso directo por índice
- Tiempo: O(1)

Ejemplo: Búsqueda

- Lineal: $O(n)$
- Binaria: $O(\log n)$

Ejemplo: Búsqueda Lineal en C

```
int busquedaLineal(int arr[], int n, int x) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == x)
            return i;
    }
    return -1;
}
```

- Complejidad: O(n)
- Caso peor: elemento no está

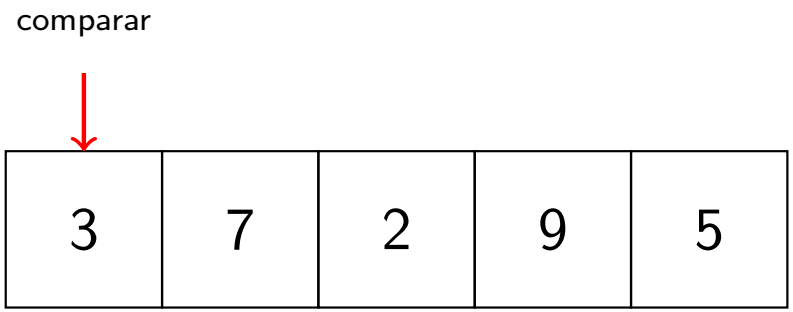
Ejemplo: Búsqueda Binaria en C

```
int busquedaBinaria(int arr[], int low, int high, int x) {
    if(low <= high) {
        int mid = (low + high) / 2;

        if(arr[mid] == x)
            return mid;
        else if(arr[mid] > x)
            return busquedaBinaria(arr, low, mid-1, x);
        else
            return busquedaBinaria(arr, mid+1, high, x);
    }
    return -1;
}
```

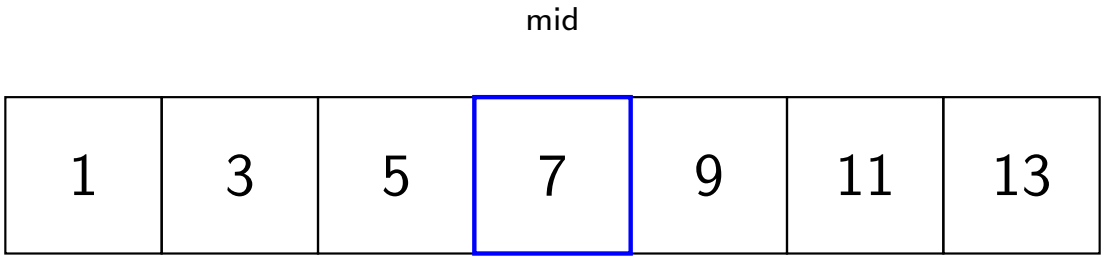
- Complejidad: O(log n)

Búsqueda Lineal



- Se revisa elemento por elemento
- Complejidad: O(n)

Búsqueda Binaria



- Divide el problema en mitades
- Complejidad: O(log n)

Diseño de Algoritmos

- Fuerza bruta
- Divide y vencerás
- Programación dinámica

Dividir y Conquistar

- Divide el problema
- Resuelve subproblemas
- Combina resultados

Ejemplo: Merge Sort

- Divide arreglo en dos
- Ordena recursivamente
- Mezcla resultados

Complejidad Merge Sort

- Relación de recurrencia:

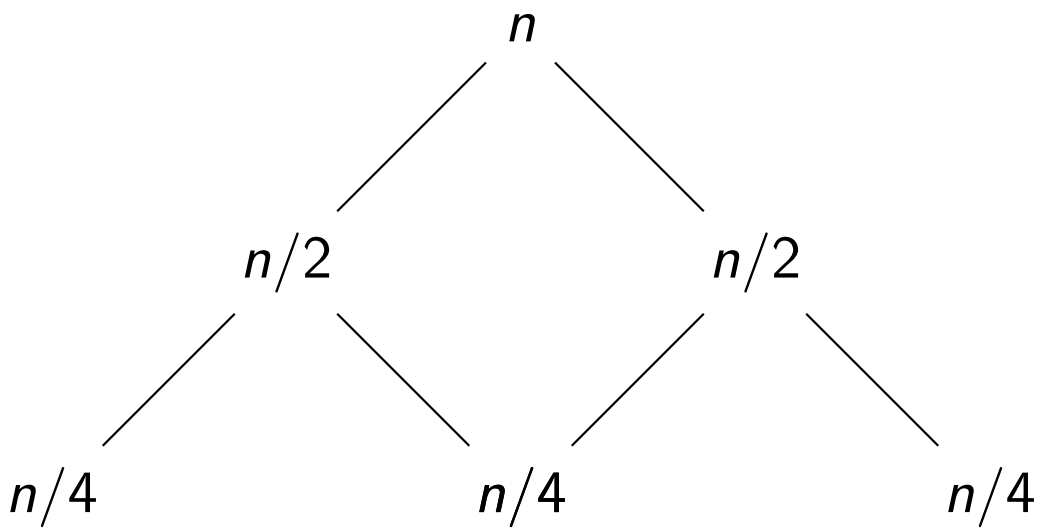
$$T(n) = 2T(n/2) + n$$

- Complejidad: $O(n \log n)$

Árbol de Recursión

- Visualiza el costo
- Niveles: $\log n$
- Costo total: $n \log n$

Árbol de Recursión (Merge Sort)



- Altura del árbol: $\log n$
- Costo por nivel: n
- Total: $n \log n$

Master Theorem

- Resuelve recurrencias
- Forma:

$$T(n) = aT(n/b) + f(n)$$

Caso 1 Master Theorem

- Si $f(n)$ es menor
- Resultado: $O(n^{\log_b a})$

Caso 2 Master Theorem

- Si $f(n)$ es igual
- Resultado: $O(n^{\log_b a} \log n)$

Caso 3 Master Theorem

- Si $f(n)$ es mayor
- Resultado: $O(f(n))$

Ventajas de Dividir y Conquistar

- Reduce complejidad
- Facilita paralelización
- Mejora rendimiento

Limitaciones

- Overhead recursivo
- Uso de memoria
- No siempre óptimo

Aplicaciones

- Ordenamiento
- Búsqueda
- Procesamiento de imágenes

Conclusiones

- El análisis de complejidad es clave
- Divide y vencerás es fundamental
- Elegir el algoritmo correcto impacta el rendimiento